Benchmark Report

Group 11

1 Benchmark methodology

1.1 Setup

The Spark cluster consisted of one Spark master and two Spark worker containers. Each worker was configured with four CPU cores and 4GB of memory, giving the cluster eight configured worker cores and 8GB of worker memory. The benchmark notebooks ran in a Jupyter container. Delta tables were stored in a MinIO object store and accessed through the S3A connector.

1.2 Optimization

Each experiment compared a baseline configuration with one changed setting. Caching was evaluated by comparing uncached and cached inputs. Partition pruning was evaluated using February 2024 queries on an unpartitioned table and a Gold table partitioned by pickup year-month. Both versions used the same date filter. Broadcast joins were evaluated by comparing a normal join with a forced broadcast join. AQE was evaluated by running the same query with AQE disabled and enabled.

Caching, partition pruning, and AQE were evaluated for Queries 1-6. Broadcast joins were evaluated for Queries 1-3.

1.3 Measurement and evaluation metrics

Each query was executed once as a warm-up before timing began. Execution time was measured using `perf_counter()` while Spark executed and collected the query result. Each configuration was measured nine times, and the median execution time was reported.

Performance was compared using the speedup factor, calculated as baseline median time divided by optimized median time. The results of each baseline and optimized query were verified to be equal before comparing their timings.

2 Execution times before and after optimization

2.1 Execution time comparison

Table 1: Execution times before and after each optimization.

Query	Optimization	Baseline (s)	Optimized (s)	Speedup
Caching				
Query 1	Caching	1.52	0.51	2.96×
Query 2	Caching	1.32	0.37	3.59×
Query 3	Caching	2	0.6	3.33×
Query 4	Caching	1.26	0.42	2.97×
Query 5	Caching	1.96	0.82	2.37×
Query 6	Caching	0.95	0.33	2.93×
Partition pruning				
Query 1	Partition	0.89	0.96	0.93×
Query 2	Partition	0.83	0.81	1.01×
Query 3	Partition	0.65	0.58	1.11×
Query 4	Partition	0.68	0.6	1.14×
Query 5	Partition	0.98	0.85	1.15×
Query 6	Partition	0.69	0.67	1.03×
Broadcast join				
Query 1	Broadcast join	4.53	2.26	1.99×
Query 2	Broadcast join	4.37	1.63	2.68×
Query 3	Broadcast join	10.45	7.78	1.34×
Adaptive Query Execution (AQE)				
Query 1	AQE enabled	1.93	1.12	1.74×
Query 2	AQE enabled	2.04	1.53	1.33×
Query 3	AQE enabled	1.57	1.21	1.30×
Query 4	AQE enabled	1.47	1.29	1.13×
Query 5	AQE enabled	1.68	1.87	0.90×
Query 6	AQE enabled	0.93	1.1	0.84×

For queries 1 and 6, an extra caching case was also considered (besides the abovementioned one). More specifically, because both queries use a common *groupby* clause, we can benefit from caching an intermediate table of the aggregated (grouped) result. To achieve that, we cache the result of grouping by month and zone (and computing demand counts), and using that for queries 1 and 6. Query 1 is just the result of the cached table, while query 6 is the result of summing the demand for all taxi zones, in each month group (from the cached table). The respective results of this extra caching procedure are shown in the following table:

Table 2: Execution times for queries 1 and 6 under each caching strategy.

Query	Baseline	Col-Cache	Co-Cache	Improvement (Col vs Co)
Query 1	1.52s	0.51s	0.13s	0.385s
Query 6	0.95s	0.33s	0.13s	0.198s

As we can see, by caching an itermediate result, we can optimize the query time even further (but of course create some extra overhead during cache creation).

2.2 Results discussion

Caching produced the most consistent improvements across all queries, with speedups from 2.37× to 3.59×. The largest caching improvement was observed for Query 2 (3.59×), while Query 5 received the smallest, but still substantial, improvement (2.37×).

Broadcast joins also improved all tested queries, particularly Query 2 (2.68×) and Query 1 (1.99×). AQE improved Queries 1-4, with the largest improvement for Query 1 (1.74×), but slightly slowed Queries 5 and 6. Partition pruning had only a limited effect: Queries 2-6 improved by at most 1.15×, while Query 1 became slower. This may be because the benchmark dataset covers only three months, so filtering to February removes a limited amount of data.

3 Analysis of query execution plans with explain formatted

The full `EXPLAIN FORMATTED` outputs for every experiment are available in the optimization notebooks included in the source-code submission.

3.1 Caching

Query 3 was used to inspect the effect of caching. In the uncached plan, Spark reads the required columns from the Gold table using `Scan parquet`. After caching a reduced view of the input table, the plan instead contains `Scan In-memory table` and `InMemoryRelation`. This confirms that the query reads the cached data during execution. Operations such as aggregation, sorting, and the broadcast exchange remain in the plan.

The median execution time of Query 3 decreased from 2.00 seconds to 0.60 seconds (3.33× speedup). Caching improved all queries, with speedups between 2.37× and 3.59×. However, materializing the cache required 5.07 seconds, so caching is most useful when the same data is reused by multiple queries.

3.2 Partition pruning

Query 5 was used to inspect partition pruning. In the partitioned plan, `Scan parquet` contains `PartitionFilters` for February 2024. This allows Spark to skip partitions outside the requested month. In contrast, the unpartitioned plan contains the February filter only as a `PushedFilters` condition, so it cannot

avoid scanning irrelevant data files. The median execution time of Query 5 decreased from 0.98 seconds to 0.85 seconds, giving a 1.15× speedup.

The effect was limited overall: Queries 2–6 improved by only 1.01× to 1.15×, while Query 1 was slightly slower (0.93×). Although the partitioned plans contain `PartitionFilters`, the dataset covers only three months, so selecting February removes a limited amount of data. The saving from pruning was therefore small and could not consistently outweigh execution overhead.

3.3 Broadcast joins

Query 2 was used to inspect the broadcast-join optimization. To ensure a fair comparison, AQE and automatic broadcasting were disabled in both runs. The baseline plan uses `SortMergeJoin`, which requires sorting and shuffling both join inputs. In the optimized plan, Spark uses `BroadcastExchange` to distribute the small weather-condition table to the workers, followed by `BroadcastHashJoin`. This avoids shuffling the large taxi-trip input.

For Query 2, the median execution time decreased from 4.37 seconds to 1.63 seconds, resulting in a 2.68× speedup. Broadcast joins also improved Queries 1 and 3 by 1.99× and 1.34×, respectively. The results confirm that broadcasting is effective when one side of the join is sufficiently small.

3.4 Adaptive Query Execution

Query 1 was used as an example where AQE improved performance. Without AQE, Spark decides the shuffle partitions before executing the query. The baseline plan therefore contains fixed `Exchange` operators.

With AQE enabled, Spark first observes the amount of intermediate data produced during execution. The `AdaptiveSparkPlan` and coalesced `AQEShuffleRead` operators show that Spark combined small shuffle partitions into fewer tasks. This reduces the overhead of scheduling tasks that process only a small amount of data. As a result, the median execution time of Query 1 decreased from 1.93 seconds to 1.12 seconds, giving a 1.74× speedup.

However, the same plan changes did not always improve execution time. Query 5 also used coalesced `AQEShuffleRead` operators, but its execution time increased from 1.68 seconds to 1.87 seconds (0.90×).

4 Discussion of performance

4.1 Which optimization produced the largest performance improvement?

Caching produced the largest performance improvement. Query 2 achieved the highest measured speedup, decreasing from 1.32 seconds to 0.37 seconds (3.59×). Caching was also the most consistent optimization, improving all six queries by between 2.37× and 3.59×.

The execution plan explains this result: after caching, Spark reads the required input through `Scan In-memory table` and `InMemoryRelation`, rather than repeatedly reading it through `Scan parquet`. This reduces repeated disk access, although the cache must first be materialized. Therefore, caching is most beneficial when the same input data is reused by several queries.

4.2 Which optimization had little or no effect? Why?

Partition pruning had the smallest overall effect. Its speedups ranged from 0.93× for Query 1 to 1.15× for Query 5. Although the partitioned plans contained `PartitionFilters`, the dataset covered only three months and the queries selected February only. Therefore, pruning removed a limited amount of data and did not substantially reduce execution time.

AQE also had limited or negative effects for some queries. It slowed Query 5 from 1.68 seconds to 1.87 seconds (0.90×) and Query 6 from 0.93 seconds to 1.10 seconds (0.84×). In these cases, the overhead of adapting the execution plan was larger than the benefit of combining small shuffle tasks.

4.3 Which queries remain computationally expensive?

Query 3 remained the most expensive query in the broadcast-join experiment, taking 7.78 seconds even after optimization. It joins the large taxi-trip table with hourly air-quality data and then performs aggregations to calculate hourly demand and the demand–air-quality relationship. Broadcasting removes the shuffle of the small table, but the large taxi-trip scan and aggregation remain necessary.

Query 5 also remained relatively expensive, taking 1.87 seconds with AQE enabled. It groups trips by weekday and hour, ranks the resulting groups, and sorts the final result. These aggregation, ranking, and sorting operations require shuffle work that cannot be removed by AQE.

Execution times should be compared mainly within each optimization experiment, since the broadcast-join experiments use source tables while most other experiments use the integrated Gold table.

4.4 What characteristics of the data explain these results?

The integrated Gold table contains millions of taxi-trip records, while the weather, taxi-zone, and hourly air-quality tables are much smaller. This size difference explains why broadcast joins were effective, the small table could be sent to each worker without shuffling the large taxi-trip table.

The Gold table covers only three months and is partitioned by month. As a result, filtering to February removes only two months of data, which explains the limited benefit of partition pruning. Caching was effective because the queries repeatedly read the large Gold table, while the cached view contains only the columns required by the queries.

Queries with joins, aggregations, ranking, and sorting remained more expensive because these operations require processing and redistributing large numbers of taxi-trip records. In particular, broadcasting a small table cannot remove the scan and aggregation work on the large table.

4.5 If the municipality expanded the platform to process data from ten cities, what changes would you recommend?

For a ten-city platform, the Gold table should be partitioned by city and pickup month. This would allow queries for a single city or time period to read only the

required partitions. Partition pruning had limited benefits for the current three-month dataset, but it would become more valuable as the amount of historical data increases. Small reference tables, such as taxi-zone, weather, and hourly aggregated air-quality data, should remain eligible for broadcast joins when they are small relative to the taxi-trip table.

AQE should be evaluated for each workload, since it improved some queries but slowed others. Caching should be reserved for frequently reused data that fits in memory, as it produced the largest speedups in the benchmark but requires cache materialization and memory resources.

The Spark cluster should scale horizontally by adding workers and allocating sufficient CPU and memory resources. Analytical data products should be updated only when new data arrives and organized by city and month, rather than being recomputed for all cities.